function interfaces with. By using `self.stdout` and `self.stderr`, your tests can capture all of your command's output to a stream and then validate the stream's contents.

Consider testing the `musicians` command: first, you you need to create at least one `Musician` object during setup. Then, you'll need a test method that runs `call_command()`, a stream to capture the output, and assertion code that validates the contents of the stream. The following listing shows exactly that.

Listing 9.4 Test code for the `musicians` management command

```
# bands/tests.py
import io
from django.core.management import call_command
...
class TestMusiciansCommand(TestCase):
    def setUp(self):
        self.musician = Musician.objects.create(
            first_name="First",
            last_name="Last",
            birth=date(1900, 1, 1)
        )

    def test_command(self):
        output = io.StringIO()

        call_command("musicians", stdout=output)
        self.assertIn("First", output.getvalue())
```

Create a Musician object in the test database.

Use a string stream to capture STDOUT.

The call_command() function takes the name of a command and optionally redirects STDOUT to a stream.

Assert that the musician's name ("First") is in the stream.

With the new test case created, you now run the `test` management command, which you learned about in chapter 8.

This example only invokes the simplest version of `musicians`, without any extra arguments. The `call_command()` function takes optional parameters, specifying the same kinds of positional arguments and flags that are used on the command-line. You can add `--last_name B` as parameter to `call_command()`, or if you prefer, you can also specify it as `last_name="B"`. To write a test case that checks all the functionality in `musicians`, you need a few more `Musician` objects and more invocations checking all flag combinations. I'll leave that to you to play with.

WARNING If your test invokes `call_command()` multiple times, don't forget to create a new `StringIO` buffer each time; otherwise, you'll be including old results in your output

9.5 Exercises

1. Create a `venues` management command that lists all the `Venue` objects in your database. Include a `--rooms` flag to optionally display the names of the rooms associated with a venue. Hint: the `action="store_true"` parameter to `add_arguments()` changes your flag, so it acts as a Boolean flag and doesn't expect any additional arguments.